

Part 7 - Manual Code Review

This review was done by reading the source directly, without looking at the Pylint findings first, then cross-checked against `intial_pylint_detailed_report` afterward to see which issues Pylint had *not* surfaced. Six problems are documented below; **#1, #2, #3, and #5 were not reported anywhere in the Pylint output**, which is more than the required minimum of two.

1. Broken password-reset flow

File: `eduplatform/services/auth_service.py`

```
def reset_password_request(self, email: str) -> bool: user =
self.user_repo.get_by_email(email) if user: reset_token =
f"reset_{len(str(hash(str(datetime.now()))))[-10:]}" ... self.notification_repo.create_notification(
recipient_id=email, ... ) return True def reset_password(self, token: str, new_password: str) ->
bool: if not token.startswith('reset_'): return False user_email = token.split('_')[-1] # <-- assumes
the email is embedded in the token user = self.user_repo.get_by_email(user_email) ...
```

Problem: `reset_password_request` never embeds the user's email (or any identifier tied to the user) into the token, the token content is just digits derived from `hash(datetime.now())`. `reset_password` then tries to recover an email address by taking everything after the last underscore in the token. This can never succeed: `token.split('_')[-1]` will always yield a numeric string, never a real email, so `user_repo.get_by_email(...)` will always return `None` and the reset always fails. (Separately, this line also crashes outright `len(...)` returns an `int`, which isn't subscriptable, but that specific crash was flagged by Pylint as `E1136 unsubscriptable-object`; the deeper design flaw, that even a fixed version of this code could never work was not.)

Quality impact: This is a completely non-functional feature hiding behind plausible-looking code. It would pass a superficial read-through and even a naive test that only checks return types, but it can never let a real user reset their password. It also reveals a security anti-pattern: reset tokens generated from a low-entropy, guessable seed (current timestamp hash) rather than a cryptographically random value tied to the user and stored with an expiry.

Recommendation: Generate a cryptographically secure random token with `secrets.token_urlsafe(32)`, store it (e.g. in a `password_resets` table/repository keyed by user id, with an expiry timestamp), and have `reset_password` look up the `token` directly rather than trying to decode an email out of it.

2. Magic number silently corrupts a statistic

File: `eduplatform/services/assignment_service.py`, `get_teacher_assignments`

```
result.append({
    ...
    'total_submissions': total_submissions,
    'graded_submissions': graded_submissions,
    'submission_rate': (total_submissions / 25) * 100, # Assuming 25 students per class
    ...
})
```

Problem: The submission rate is computed against a hardcoded assumption of exactly 25 students per class, for every class, regardless of its actual size. A class of 30 students with 100% submission would report a 120% "submission rate"; a class of 10 students would be capped at 40% even if everyone submitted.

Quality impact: This isn't a style nit, the number produced is simply wrong for any class that doesn't happen to have exactly 25 students, and nothing in the code or its output signals that the value is unreliable. Pylint has no way to catch this because `25` is a syntactically ordinary literal; only a human reading the comment and the surrounding logic notices the assumption doesn't hold.

Recommendation: Look up the actual class roster size (via a class/enrollment repository) and use that as the denominator: `(total_submissions / class_size) * 100` with a guard for `class_size == 0`. If enrollment data genuinely isn't available yet, the field should be omitted or explicitly marked as an estimate rather than presented as a real statistic.

3. Duplicated logic within a single file

File: `eduplatform/services/export_service.py`

`export_user_data`, `export_class_data`, and `export_school_data` each end with essentially the same ~25-line block:

```
result = {}
for data_type, data in export_data.items():
    if not data:
        continue
    filename = f"{base_filename}_{data_type}.{format}"
    output_path = os.path.join(output_dir, filename)
    try:
        exported_path = self.export_utils.export_data(
```

```

        data=data, output_path=output_path, format=format,
        sheet_name=data_type.replace('_', ' ').title()
    )
    result[data_type] = exported_path
except Exception as e:
    print(f"Warning: Failed to export {data_type}: {str(e)}")
    continue

manifest = {'export': {..., 'file_paths': result}}
manifest_path = os.path.join(output_dir, f"{base_filename}_manifest.json")
with open(manifest_path, 'w') as f:
    import json
    json.dump(manifest, f, indent=2)
result['manifest'] = manifest_path
return result

```

Problem: This is copy-pasted three times with only the manifest's inner fields changing.

Quality impact: Any fix to the export loop (say, retrying failed exports, or writing errors to the manifest instead of just printing them) has to be made in three places, and it's easy to update two and miss the third, a classic maintenance trap. It's also a low-cohesion smell: the class is really doing "build a dataset" + "write dataset(s) to disk" + "write a manifest," and those last two steps deserve to be factored out.

Recommendation: Extract a single private helper, e.g. `_export_and_manifest(self, data_by_type, output_dir, base_filename, format)`, that all three public methods call with just their prepared dictionary.

4. Duplicated logic between methods, not just within one method

File: `eduplatform/services/grade_service.py`

`get_student_grades` (lines 242–261) and `get_class_grades` (lines 297–315) each build this same 14-key dictionary from a `Grade` object:

```

result.append({
    'id': grade._id, 'subject': grade._subject, 'type': grade._type.value,
    'score': grade._score, 'max_score': grade._max_score,
    'percentage': grade.percentage, 'letter_grade': grade.letter_grade,
    'gpa_points': grade.gpa_points, 'comments': grade._comments,
    'teacher_id': grade._teacher_id,
    'teacher_name': teacher._full_name if teacher else 'Unknown',
    'assignment_id': grade._assignment_id,

```

```
'created_at': grade._created_at.isoformat(),
'updated_at': grade._updated_at.isoformat() if grade._updated_at else None
})
```

Problem: Identical serialization logic, duplicated instead of shared. As with #3, Pylint's duplicate-code detector didn't flag this pairing either.

Quality impact: If a new field is added to grades (e.g. a rubric reference), a developer has to remember to update both call sites; missing one produces silently inconsistent API responses between "my grades" and "class grades" views.

Recommendation: Extract a `_serialize_grade(self, grade, teacher)` helper and call it from both methods.

5. Two parallel, inconsistent grade-tracking systems (architecture level not caught by Pylint)

Files: `eduplatform/models/student.py` vs.
`eduplatform/services/grade_service.py` + `eduplatform/models/grade.py`

student.py: Student keeps its own private grade store
`self._grades: Dict[str, List[Dict]] = {}`

```
def receive_grade(self, assignment_id: str, grade: int, teacher_id: str, comment: str = "") -> bool:
    ...
    self._grades[subject].append({'value': grade, 'date': ..., 'teacher_id': teacher_id, ...})
```

versus

```
# grade_service.py: the "real" grading system used everywhere else
def record_grade(self, student_id, subject, grade_type, score, teacher_id, ...):
    grade = Grade(student_id=student_id, subject=subject, ...)
    self.grade_repo.add(grade)
```

Problem: The codebase has two independent representations of "a student's grades": a plain dict living inside the `Student` object (`_grades`, populated only via `Student.receive_grade`), and the proper `Grade` model persisted through `GradeRepository/GradeService` (used by `AssignmentService.grade_assignment`, `ExportService`, and reporting). Nothing keeps them in sync, nowhere does `AssignmentService` or `GradeService` call `Student.receive_grade`, and nothing calls `GradeService.record_grade` from `Student`. This is a case of low cohesion at the domain-model level: the concept "grade" is duplicated instead of owned by one component.

Quality impact: `student.view_grades()` and `student.calculate_overall_average()` will return empty/zero for every grade that was actually recorded through the normal `GradeService` flow that the rest of the application uses, while grades placed via `Student.receive_grade` never show up in report cards, exports, or class statistics. This is a real data-integrity bug that unit tests targeting either half of the system in isolation would never catch, and it's invisible to a linter because both halves are individually well-formed code, the flaw only exists in the relationship between two files.

Recommendation: Delete the `_grades/receive_grade/view_grades/calculate_overall_average` machinery from `Student` and route every read of "this student's grades" through `GradeService`, which already owns persistence, notifications, and GPA/average calculations for grades.

6. Debug `print()` statements used as production logging, with sensitive data exposure

File: `eduplatform/services/auth_service.py`, `register_user` (and repeated in `export_service.py`)

```
print(f"[AUTH] register_user called with: {user_type}, {full_name}, {email}, {kwargs}")
...
print(f"[AUTH] Set phone: {kwargs['phone']}")
print(f"[AUTH] Set address: {kwargs['address']}")
...
except Exception as e:
    print(f"[AUTH] Error creating user: {str(e)}")
    print(f"[AUTH] Exception type: {type(e).__name__}")
    raise
```

Problem: There are over a dozen `print()` calls acting as the only "logging" in `register_user`. They can't be turned off, filtered by severity, or routed to a log aggregator, and more importantly they print personal data (full name, email, phone, address, and the entire `kwargs` dict, which could include the plaintext password if a caller ever passed it through `kwargs`) straight to stdout. Pylint has no rule against `print()` usage and never mentions logging practice anywhere in the report, so this is invisible to the automated scan entirely.

Quality impact: In any real deployment this is both a debugging/operability problem (no log levels, no structured fields, can't be disabled in production) and a potential privacy/security issue (PII and exception internals dumped unconditionally to console/log files that may not be access-controlled the way an audit log would be).

Recommendation: Replace all `print()` calls with the standard `logging` module (`logger.debug(...)`, `logger.info(...)`, `logger.exception(...)`), configured with an appropriate level per environment, and stop logging raw `kwargs` or other unfiltered user-supplied dictionaries log only the specific fields that are safe and useful (e.g. user id, not password-adjacent payloads).
